

학습목표

1. JPA 등장배경
2. JPA
3. Spring Data JPA

-JPA등장 배경

JDBC(Java DataBase Connectivity)는 Java에서 데이터베이스에 접속하고 데이터 처리 구현에 사용 되는 Java API 이다.

JDBC의 장점은 각각의 DBMS가 제공하는 JDBC 드라이버를 이용하여 DBMS의 종류와 상관없이 하나의 JDBC API를 이용하여 DB 작업을 처리할 수 있다는 것이다.

그러나, JDBC만을 이용하는 경우

- 1) 간단한 SQL을 실행함에도 **중복된 코드를 반복적으로 사용하는** CRUD작업
- 2) DB에 따라 일관성 없는 정보를 가진 채로 Checked Exception(SQLException) 처리
- 3) 객체지향적 개발보다 SQL중심적인 개발의 문제점
- 4) SQL 을 확인하기 전까지 엔티티를 신뢰 할 수 없는 단점
- 5) Java와 DB사이에게 개발자가 직접 매핑 해야 하는 번거로움.

그래서

이런 복잡하고 번거로운 작업을 해결하기 위해 데이터베이스와 연동되는 시스템을 빠르게 개발할 수 있는 **Persistence Framework**를 사용하기 시작하였다.

SQL Mapper을 이용한 영속 계층 SQL Mapper는 객체와 SQL을 매핑하여 데이터를 객체화 하는 Persistence Framework 이다. 즉, **테이블과 객체 간의 관계를 매핑하는 것이 아닌 직접 작성한 SQL문의 결과와 객체의 필드를 매핑하는 것이 메인 컨셉이다** (ex)MyBatis

SQL Mapper를 이용하면 기존에 JDBC 만을 이용했을 때보다 코드가 간결해 지고 유지보수성이 향상된다.

그러나

SQL Mapper가 JDBC 만을 사용했을 때보다 많은 불편함을 해소해 주었음에도 여전히 문제가 있다.

- SQL을 직접 작성하기에 특정 DB에 종속적
- 비슷한 CRUD SQL 작성 및 DAO를 반복적으로 개발
- 테이블 필드 변경 시 유지보수의 어려움

즉, 코드상에서는 SQL과 JDBC API를 분리했어도 논리적으로는 강한 의존성을 가지고 있게 되며 SQL에 의존적인 개발을 해야 하는 문제는 여전히 발생한다.

ORM을 이용한 영속 계층 ORM은 객체와 관계(테이블)을 매핑하는 기술이다. ORM은 객체 간의 관계들을 바탕으로 SQL문을 자동으로 생성하고 직관적으로 데이터를 조작하게 하여 개발자의 불편함을 해소한다. ORM의 대표적인 기술은 바로 JPA가 있다. 원래 JPA 기술이 나오기 전까지는 EJB(Entity Bean)이라는 Java 표준 기술이 있었다. 하지만 워낙 복잡하다 보니 Gavin King이라는 개발자가 하이버네이트 라는 오픈 소스를 만들어냈고 Java 진영에서 Gavin King을 영입하여 하이버네이트 기반으로 Java ORM 표준인 JPA를 만들었다.

Persistence Framework 종류

JDBC 프로그래밍에서 경험하게 되는 복잡함이나 번거로움 없이 간단한 작업만으로 데이터베이스와 연동되는 시스템을 빠르게 개발할 수 있다.

일반적으로 1)SQL Mapper와 2)ORM으로 나뉜다.

1) SQL Mapper

Database data <-> SQL Mapper(SQL) <-> Object 필드 직접 작성한 SQL 문장으로 데이터베이스 데이터를 다룬다. Ex) Mybatis, JdbcTemplate(Spring)

2) ORM

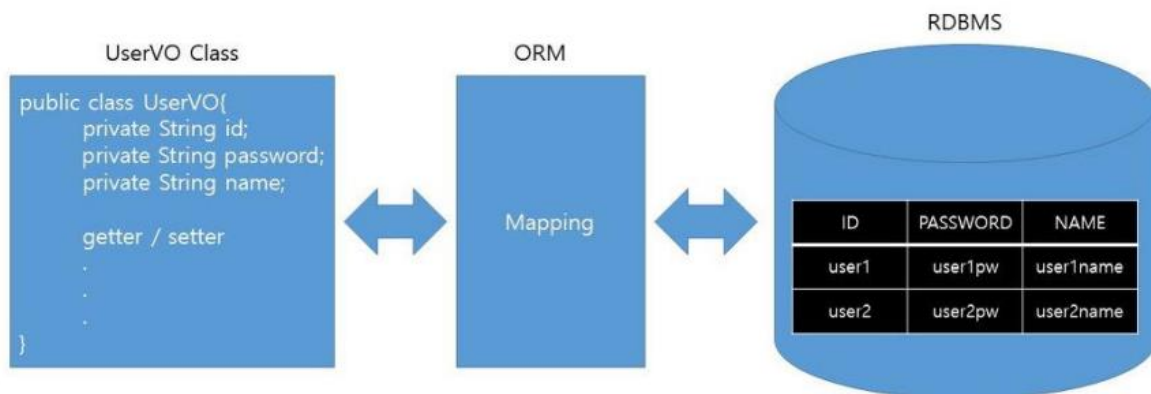
Database data <-> ORM <-> Object 필드 객체를 통해서 간접적으로 데이터베이스의 데이터를 다룬다. 객체와 관계형 데이터베이스의 데이터를 자동으로 맵핑시킨다. ex) JPA의 Hibernate 등

JPA(Java Persistence API)란?

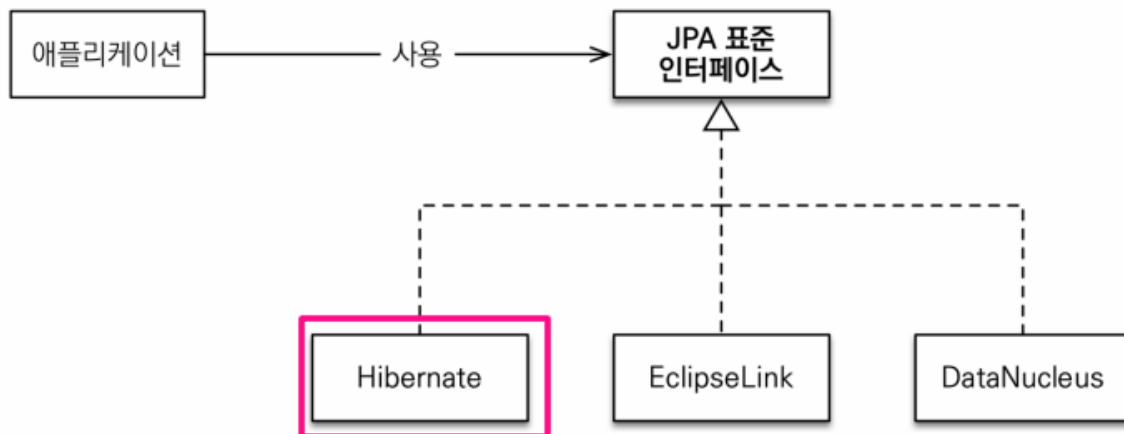
데이터베이스는 객체 구조와는 다른 데이터 중심의 구조를 가지므로 객체를 데이터베이스에 직접 저장하거나 조회할 수 없다. 따라서 개발자가 객체지향 애플리케이션과 데이터베이스 중간에서 SQL과 JDBC API를 사용해서 변환 작업 (객체 → 데이터베이스)을 직접 해주어야 한다.

JPA는 Java Persistence API 의 약자로서, RDBMS와 객체 사이의 불일치에서 오는 패러다임을 해결 하기 위해서 만들어진 ORM(Object-Relational Mapping) 기술이다. ORM이란 Object Relational Mapping, 객체-관계 매핑의 줄인 말로써 OOP의 객체 구현 클래스와 RDBMS에서 사용하는 테이블을 자동으로 매핑하는 것을 의미한다.

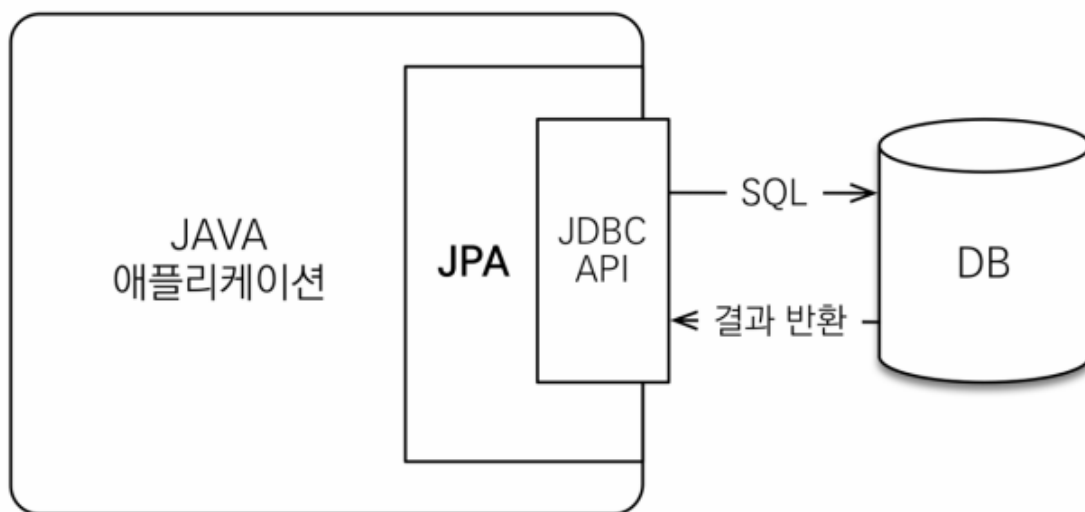
이 때, 클래스와 테이블은 기존 부터 호환 가능성을 두고 만들어진 것이 아니므로 불일치가 발생하는데 이를 ORM을 통해서 객체 간의 관계를 바탕으로 SQL문을 자동 생성하여 불일치를 해결한다. 이 방법을 통해서 SQL문을 구현 할 필요 없이 객체를 통해 간접적으로 데이터베이스를 조작할 수 있다.



JPA 는 Java ORM에 대한 API 표준 명세이고, 인터페이스의 모음이다. 따라서 구현체가 없으므로, 사용하기 위해서는 ORM프레임워크를 선택해야 한다. 다양한 프레임워크가 존재하지만 가장 대중적인 것은 하이버네이트이다



JPA는 애플리케이션과 JDBC 사이에서 동작한다. JPA 내부에서 JDBC API를 사용하여 SQL을 호출하여 DB와 통신한다. 개발자가 ORM 프레임워크에 저장하면 적절한 INSERT SQL을 생성해 데이터베이스에 저장해주고, 검색을 하면 적절한 SELECT SQL을 생성해 결과를 객체에 매핑하고 전달한다.



JPA의 장점

첫째, 생산성 JPA를 사용하면 Java 컬렉션에 객체를 저장하듯이 JPA에게 저장할 객체를 전달하면 된다. 지루하고 반복적인 코드를 개발자가 직접 작성하지 않아도 되며, DDL문도 자동으로 생성해주기 때문에 데이터베이스 설계 중심을 객체 설계 중심으로 변경할 수 있다.

둘째, 유지보수 필드를 하나만 추가해도 관련된 SQL과 JDBC 코드를 전부 수행해야 했지만 JPA는 이를 대신 처리 해주기 때문에 개발자가 유지보수해야 하는 코드가 줄어든다.

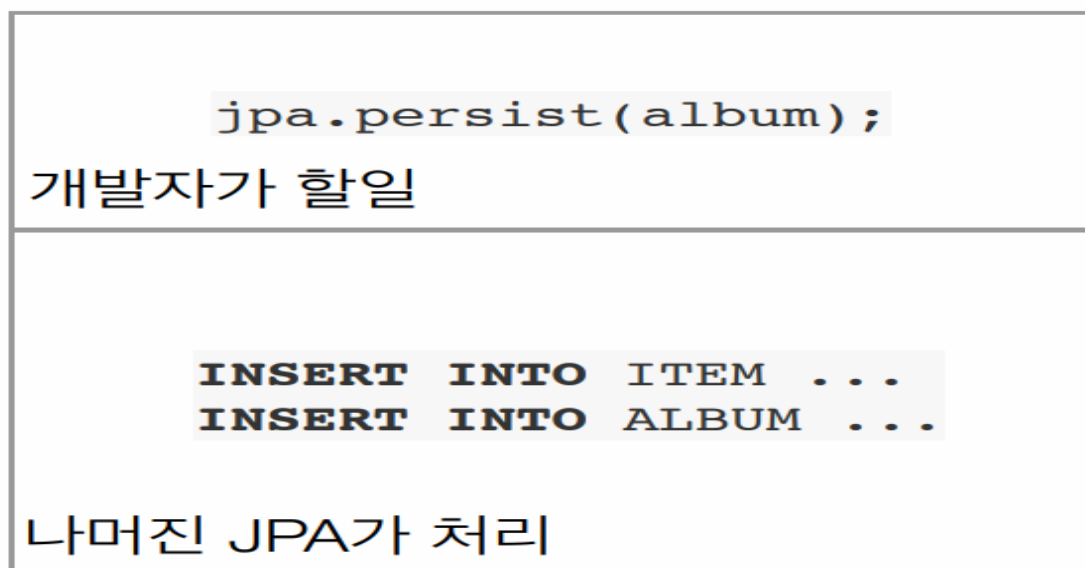
셋째, 패러다임의 불일치 해결 JPA는 연관된 객체를 사용하는 시점에 SQL을 전달할 수 있고, 같은 트랜잭션 내에서 조회할 때 동 일성도 보장하기 때문에 다양한 패러다임의 불일치를 해결한다.

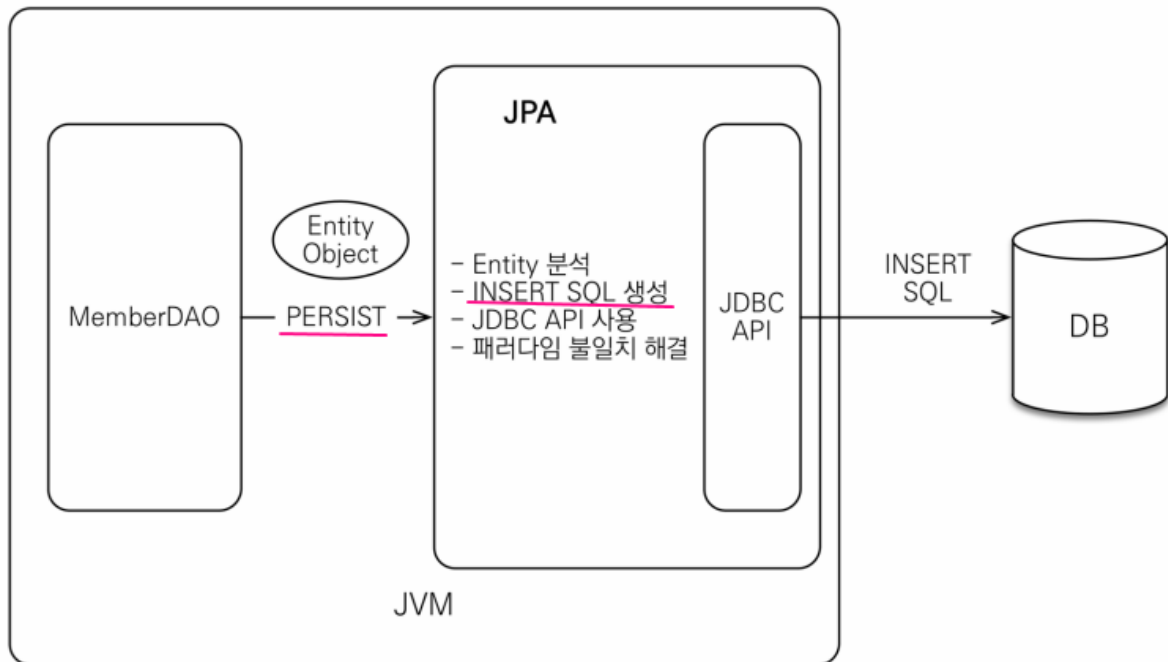
넷째, 성능 애플리케이션과 데이터베이스 사이에서 성능 최적화 기회를 제공한다. 같은 트랜잭션안에서는 같은 엔티티를 반환하기 때문에 데이터 베이스와의 통신 횟수를 줄일 수 있다. 또한, 트랜잭션을 commit하기 전까지 메모리에 쌓고 한 번에 SQL을 전송한다.

다섯번째, 데이터 접근 추상화와 벤더 독립성 RDB는 같은 기능이라도 벤더마다 사용법이 다르기 때문에 처음 선택한 데이터베이스에 종속되고 변경이 어렵다. JPA는 애플리케이션과 데이터베이스 사이에서 추상화된 데이터 접근을 제공하기 때문에 종속이 되지 않도록 한다. 만약 DB가 변경되더라도 JPA에게 알려주면 간단하게 변경이 가능하다

JPA동작

1) 저장





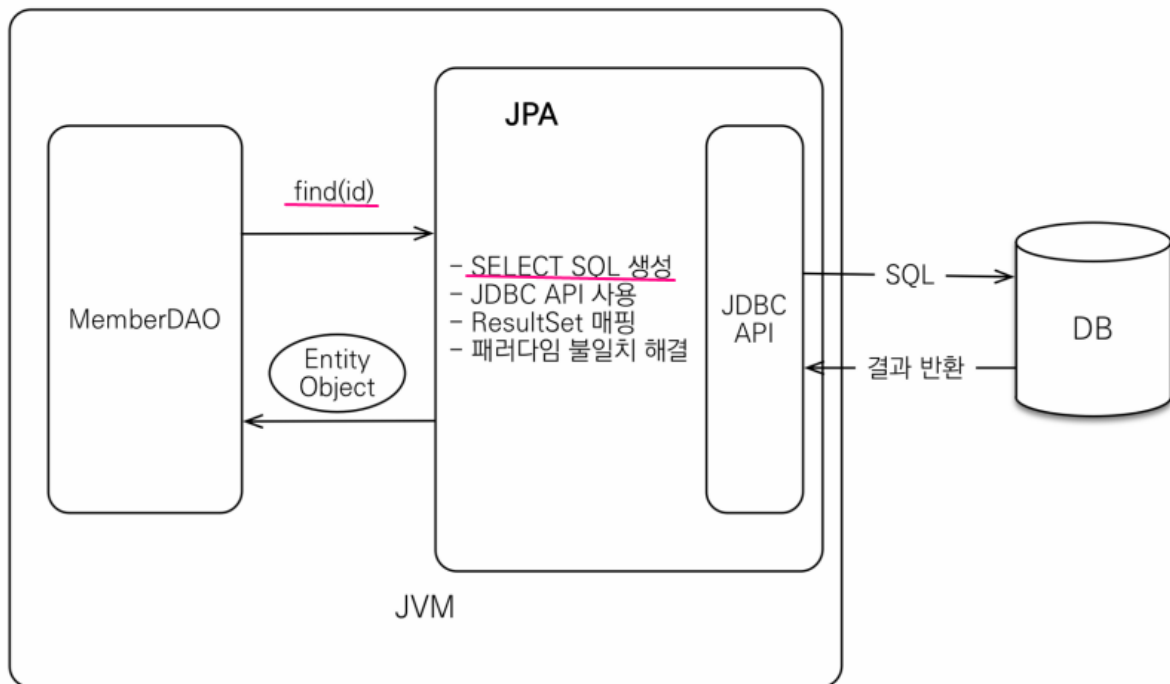
2) 조회

```
Album album = jpa.find(Album.class, albumId);
```

개발자가 할일

```
SELECT I.*, A.*
FROM ITEM I
JOIN ALBUM A ON I.ITEM_ID = A.ITEM_ID
```

나머진 JPA가 처리



- JPA 시작하기

- JPA구현체 라이브러리 준비
- 연결하려는 Database Driver 라이브러리 준비
- META-INF/persistence.xml 파일 준비 - JPA관련정보 설정파일
- 객체 매핑 클래스

- **@Entity**: JPA가 관리할 객체
- 엔티티라 한다.
- **@Id**: DB PK와 매핑 할 필드

```
@Entity
public class Member {

    @Id
    private Long id;
    private String name;
    ...
}
```

```
create table Member (
    id bigint not null,
    name varchar(255),
    primary key (id)
)
```

persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence" version="2.1">

    <persistence-unit name="hello">
        <properties>

            <!-- 필수 속성 -->
            <property name="javax.persistence.jdbc.driver" value="org.h2.Driver"/>
            <property name="javax.persistence.jdbc.user" value="sa"/>
            <property name="javax.persistence.jdbc.password" value=""/>
            <property name="javax.persistence.jdbc.url" value="jdbc:h2:tcp://localhost/~test"/>
            <property name="hibernate.dialect" value="org.hibernate.dialect.H2Dialect" />

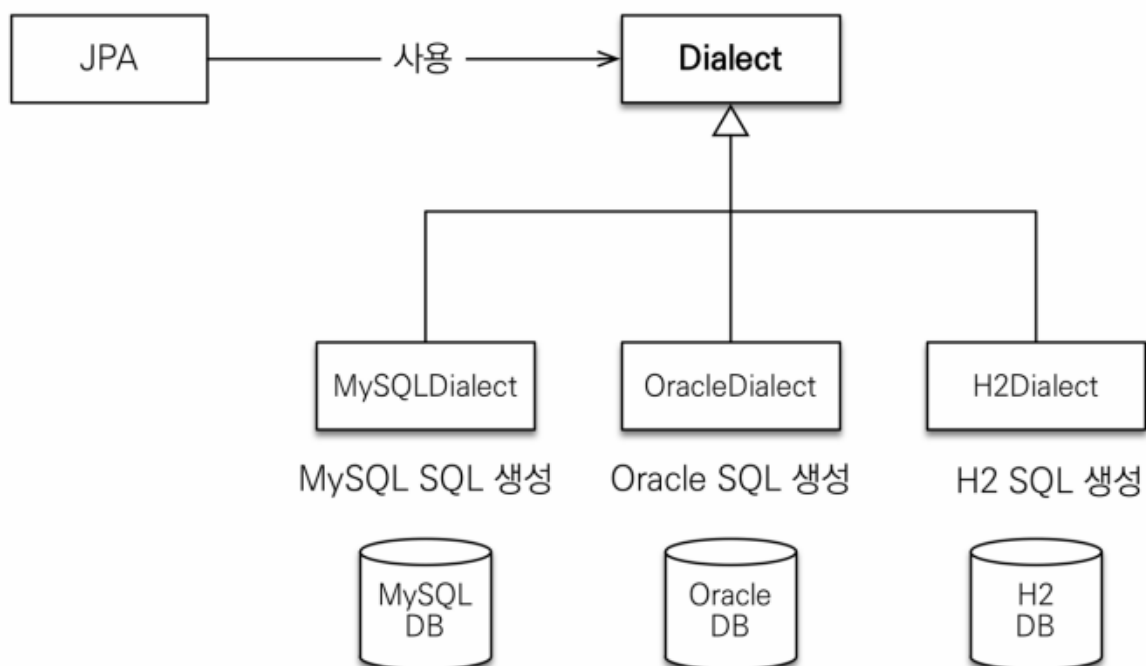
            <!-- 옵션 -->
            <property name="hibernate.show_sql" value="true" />
            <property name="hibernate.format_sql" value="true" />
            <property name="hibernate.use_sql_comments" value="true" />
            <property name="hibernate.id.new_generator_mappings" value="true" />

            <!--<property name="hibernate.hbm2ddl.auto" value="create" />-->
        </properties>
    </persistence-unit>
</persistence>
```


데이터베이스 방언(Dialect)

Dialect는 JPA를 사용하는 애플리케이션이 특정 데이터베이스와 상호작용할 수 있도록 도와주는 구성 요소이다. 데이터베이스마다 SQL 문법과 기능이 다르기 때문에, JPA는 이 차이를 처리하기 위해 각 데이터베이스에 맞는 Dialect를 제공한다.

JPA Dialect는 데이터베이스와 JPA 간의 통신을 원활하게 해주며, 데이터베이스의 특성을 반영하여 SQL 변환, 데이터 타입 매핑 및 기능 지원을 제공한다. 이를 통해 개발자는 데이터베이스의 세부 사항을 신경 쓰지 않고도 객체 지향적으로 데이터베이스와 상호작용할 수 있다.



예제)

```

26
27 @Entity //JPA가 관리해야할객체
28 @Builder
29 @ToString
30 @NoArgsConstructor
31 @AllArgsConstructor
32 @Setter
33 @DynamicUpdate
34 //@DynamicInsert
35 public class Customer {
36     @Id //pk
37     // @GeneratedValue(strategy = GenerationType.AUTO)
38     @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "customer_id_seq")
39     @SequenceGenerator(name = "customer_id_seq", sequenceName = "customer_id_seq", allocationSize = 1)
40     private Long id;
41
42     @Column(nullable = false, length = 100, name = "user_name", unique = true)
43     private String userName; //기본은 null허용
44
45     // @Transient // 생성될때 컬럼을 만들지 않는다.
46     private int age; //기본 not null
47
48     //등록일
49     @CreationTimestamp //새로운 레코드가 추가될때 날짜가 자동
50     private LocalDateTime insertDate;
51
52     //수정일
53     @UpdateTimestamp //추가, 수정될 날짜가 자동 변경된다.
54     private LocalDateTime updateDate;
55
56     //생일
57     @Temporal(TemporalType.DATE)
58     private Date birthDay;
59
60 }

```

```

1 package exam.main;
2
3 import java.util.Date;
4
5 public class MainApp {
6     public static void main(String[] args) {
7         System.out.println("***JPA 시작하기 *****");
8         EntityManagerFactory emf = Persistence.createEntityManagerFactory("JPAProject");
9         EntityManager em = emf.createEntityManager();
10        EntityTransaction et = em.getTransaction();
11        et.begin();
12
13        //저장
14        em.persist(Customer.builder().userName("희정").age(20).birthDay(new Date()).build());
15
16        //조회
17        Customer cu = em.find(Customer.class, 2L);
18        System.out.println("cu = " + cu);
19
20        et.commit();
21
22        em.close();
23        emf.close();
24        System.out.println("***JPA 끝 *****");
25    }
26 }

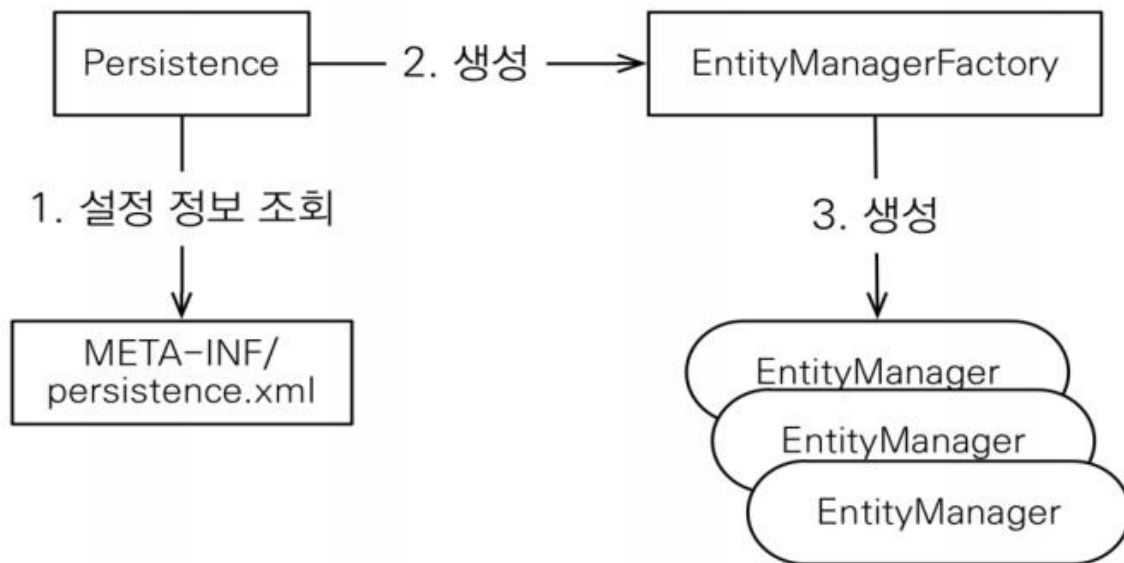
```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <persistence version="2.1" xmlns="http://xmlns.jcp.org/xml/ns/persistence" xmlns:xsi="
4 xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence http://xmlns.jcp.org/xml
5 <persistence-unit name="JPAProject">
6
7 <properties>
8
9 <!-- 필수 속성 -->
10 <property name="javax.persistence.jdbc.driver" value="oracle.jdbc.driver.OracleDri
11 <property name="javax.persistence.jdbc.user" value="c##jpa"/>
12 <property name="javax.persistence.jdbc.password" value="jpa"/>
13 <property name="javax.persistence.jdbc.url" value="jdbc:oracle:thin:@localhost:152
14 <property name="hibernate.dialect" value="org.hibernate.dialect.Oracle10gDialect"/>
15
16
17 <!-- 옵션 -->
18 <property name="hibernate.show_sql" value="true" />
19 <property name="hibernate.format_sql" value="true" />
20 <property name="hibernate.use_sql_comments" value="false" />
21 <property name="hibernate.id.new_generator_mappings" value="true" />
22 <property name="hibernate.hbm2ddl.auto" value="none" /> <!--create | create-drop |
23 </properties>
24
25 </persistence-unit>
26 </persistence>

```

EntityManagerFactory를 이용하여 EntityManager 생성



1. persistence.xml 파일을 통해서 JPA 관련 정보를 설정한다.
2. EntityManagerFactory 객체를 생성한다. persistence-unit 태그의 이름 정보 지정)
3. EntityManager 객체를 생성하여 Entity 객체를 영속성 컨텍스트(Persistence Context)를 통해 관리한다.
4. 모든 데이터 변경은 트랜잭션 안에서 이루어져야 한다.(트랜잭션을 시작하고 처리해야 함)

-EntityManagerFactory

데이터베이스와 상호 작용을 위한 EntityManager 객체를 생성하기 위해 사용되는 객체로서 애플리케이션에서 **한 번만 객체를 생성하고 공유해서 사용한다**. Thread-Safe 하므로 여러 스레드에서 시에 접근해도 안전하다. EntityManagerFactory 객체를 통해 생성되는 모든 EntityManager 객체는 동일한 데이터베이스에 접속한다.

- EntityManager

Entity 객체를 관리하는 객체이다. 데이터베이스에 대한 **CRUD 작업은 모두 영속성 컨텍스트를 사용하는 EntityManager 객체를 통해 이루어진다**. 동시성의 문제가 발생할 수 있으니 스레드 간에 공유하지 않는다

제공되는 메소드

flush()

영속성 컨텍스트(Persistence Context)의 변경 내용을 데이터베이스에 반영한다. 일반적으로는 flush() 메서드를 직접 사용하지는 않고, Java 애플리케이션에서 커밋 명령이 들어왔을 때 자동으로 실행된다.

detach()

특정 Entity를 준영속 상태(영속 컨텍스트의 관리를 받지않음)로 바꾼다.

clear()

Persistence Context를 초기화한다.

close()

Persistence Context를 종료한다.

merge()

준영속 상태의 엔티티를 이용해서 새로운 영속 상태의 엔티티를 반환한다.

find()

식별자 값(Key 필드)을 통해 Entity를 찾는다.(DB 테이블의 데이터 또는 행을 찾는다.)

persist()

생성된 Entity 객체를 영속성 컨텍스트(Persistence Context)에 저장한다.

remove()

식별자 값을 통해 영속성 컨텍스트(Persistence Context)에서 Entity 객체를 삭제한다.

Entity란?

JPA에서 엔티티란 DB 테이블에 대응하는 클래스이고 엔티티 객체는 DB 테이블의 데이터(하나의 행) 이다. @Entity 어노테이션이 붙은 클래스를 JPA에서는 엔티티라고 부르며, 이 엔티티는 영속성 컨텍 스트에 담겨 EntityManager에 의해서 관리된다.

Entity관련 주요 Annotation

@Entity

클래스 레벨에 적용한다. 해당 클래스를 테이블과 매핑한다고 JPA에게 알려주는 역할을 한다. 해당 어노테이션이 적용된 클래스를 엔티티 클래스라고 한다.

@Table

클래스 레벨에 적용한다. 엔티티 클래스에 매핑할 테이블 이름을 JPA에게 알려주는 역할을 한다. @Table(name = "테이블이름") 해당 어노테이션을 생략하면 클래스 이름에서 모든 글자를 소문자로 변경하여 테이블을 생성한다. 클래스명을 모두 소문자로 변경하고 각 워드마다 _ 를 붙인 명칭의 테이블 이름으로 매핑하도록 persistence.xml 에 설정하면 camel case 와 snake_case 간에 자동 변환된다. ex)OrderDetail --> order_detail

[persistence.xml 설정]

```
<property name="hibernate.physical_naming_strategy"
value="camel case 를 snake_case 로 변경하는 기능의 클래스"/>
```

@Id

필드 레벨에 적용한다. 엔티티 클래스의 필드를 **테이블의 기본 키 (Primary Key)에 매핑한다**. 해당 어노테이션이 적용된 필드를 식별자 필드라고 한다. - MySQL의 경우 @Id @GeneratedValue(strategy = GenerationType.IDENTITY) @Column 필드 레벨에 적용한다. 해당 필드를 테이블의 컬럼에 매핑한다. name 속성으로 매핑할 컬럼의 이름을 지정할 수 있으며 nullable, unique, length 등도 설정 가능하다.

@Column(name = "매핑할 컬럼명")

name 속성을 생략하거나 @Column 어노테이션을 생략하면 필드명을 그대로 사용해서 컬럼명으로 매핑한다. 이름이 age인 필드에 매핑 정보를 생략하면, age 컬럼으로 자동 매핑한다. 대소문자를 구분하는 DB를 사용하면 @Column 을 사용하여 명시적으로 매핑해야 한다.

@CreationTimestamp

엔티티가 처음 생성될 때의 타임스탬프를 자동으로 설정하는 것으로 주로 등록일자 같은 필드에서 사용한다. Hibernate에서 제공하는 기능으로, 객체가 처음 저장될 때 자동으로 현재 시간이 설정된다.

```
@CreationTimestamp  
private LocalDateTime createdDate;
```

@UpdateTimestamp

엔티티가 업데이트될 때의 타임스탬프를 자동으로 설정하는 것으로 주로 수정일자 같은 필드에 사용된다. 객체가 수정될 때마다 현재 시간이 자동으로 업데이트된다.

```
@UpdateTimestamp  
private LocalDateTime lastModifiedDate;
```

@Temporal

날짜 및 시간 필드의 매핑 방식을 지정한다. Java8의 java.time 패키지를 사용하는 대신, java.util.Date 또는 java.util.Calendar를 사용하는 경우에 필요하다.

@Temporal(TemporalType.DATE)는 날짜만 저장하고, @Temporal(TemporalType.TIME)은 시간만 저장하며, @Temporal(TemporalType.TIMESTAMP)는 날짜와 시간을 모두 저장한다.

```
@Temporal(TemporalType.DATE)  
private Date eventDate;
```

@transient

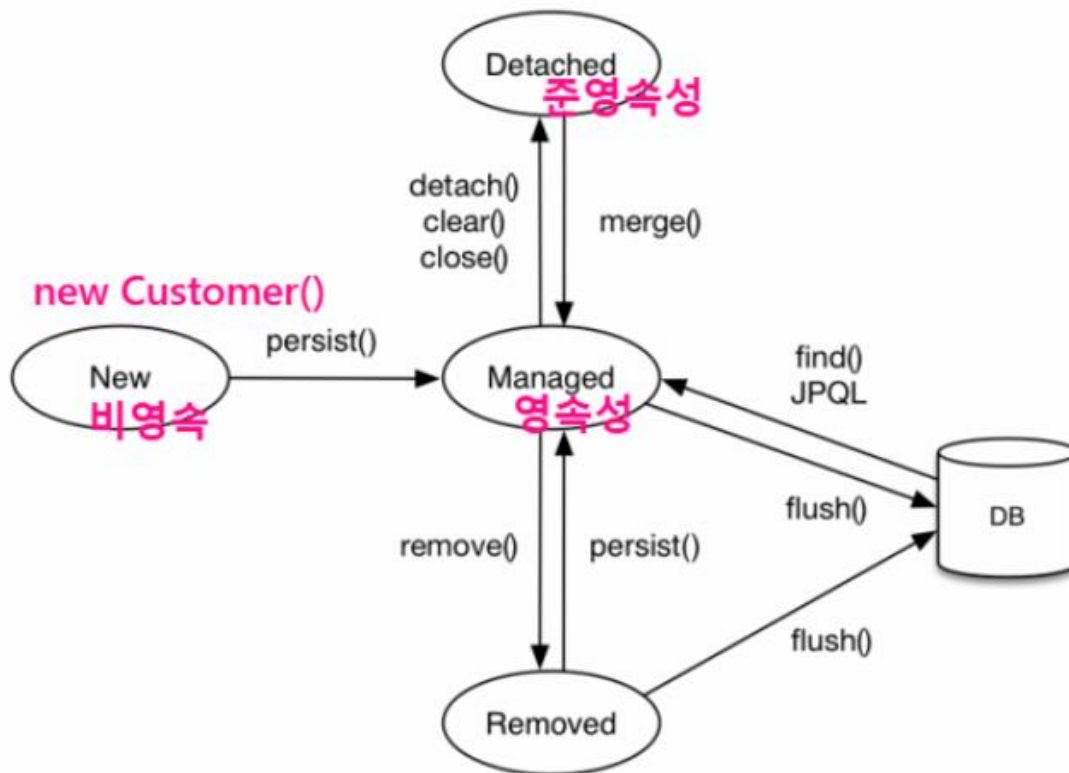
JPA에 의해 관리되지 않는 필드를 표시한다. 즉, 해당 필드는 데이터베이스에 매핑되지 않는다.

계산된 필드나 임시 데이터에 사용한다.

@Transient로 표시된 필드는 JPA가 무시하며, 데이터베이스에 저장되지 않는다.

```
@Transient  
private String temporaryField;
```

Entity의 생명주기



1. 비영속(new/transient) : 영속성 컨텍스트와 전혀 관계가 없는 상태
2. 영속(managed) : 영속성 컨텍스트에 저장된 상태
3. 준영속(detached) : 영속성 컨텍스트에 저장되었다가 분리된 상태
4. 삭제(removed) : 삭제된 상태

영속성 컨텍스트(persistence context)

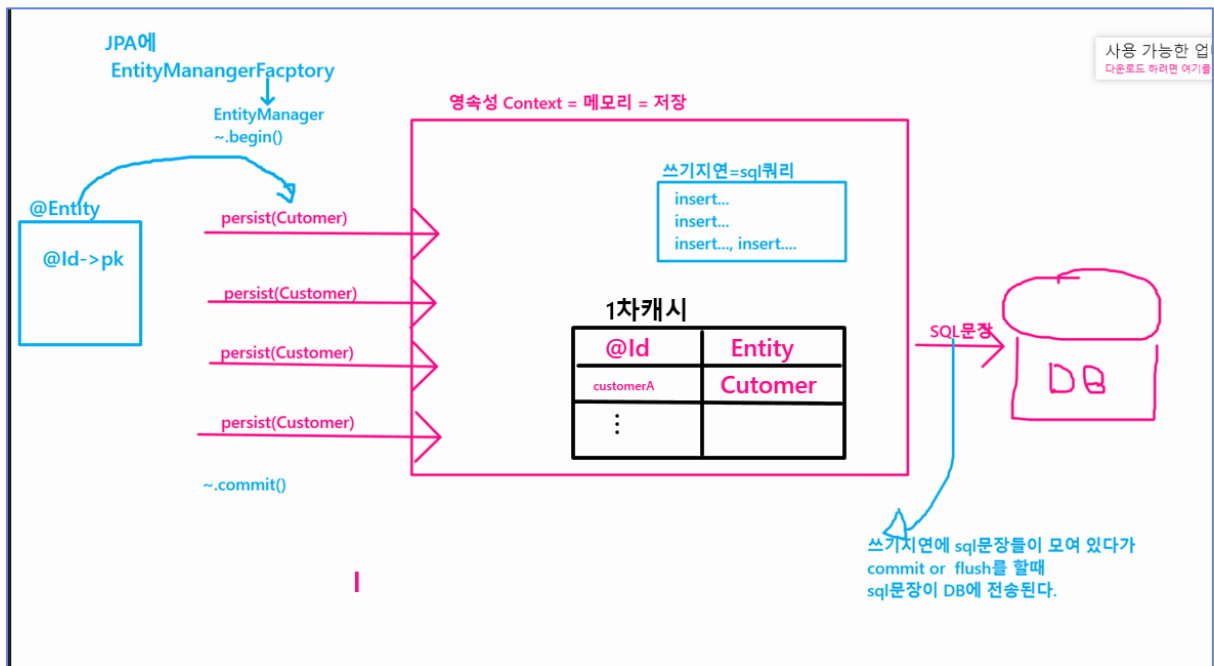
- 어플리케이션과 데이터베이스 사이에 존재하는 **논리적인 개념으로** 엔티티를 저장하고 관리하는 영역이다. EntityManager 객체를 통해서만 접근이 가능하다. 영속성 컨텍스트에 존재하는 엔티티는 **플러시 기능 처리시 데이터베이스에 반영된다.**

- 1) entityManager.flush() 로 플러시 기능을 직접 호출
- 2) 트랜잭션 커밋(commit) 시 entityManager.flush() 메서드 자동 호출
- 3) JPQL 쿼리 실행 시 entityManager.flush() 메서드 자동 호출



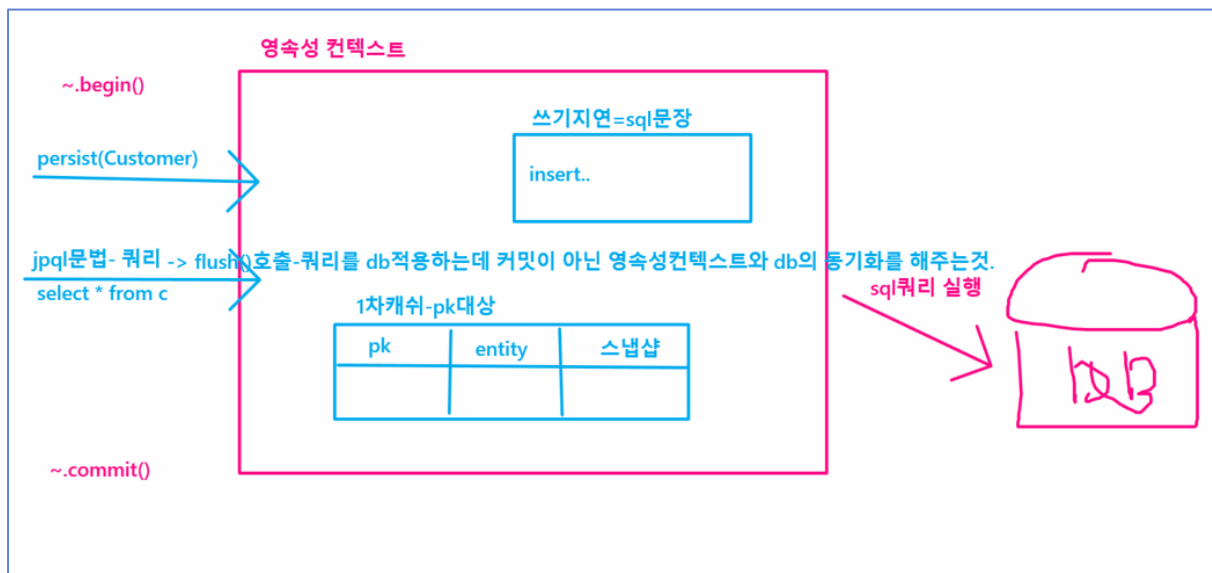
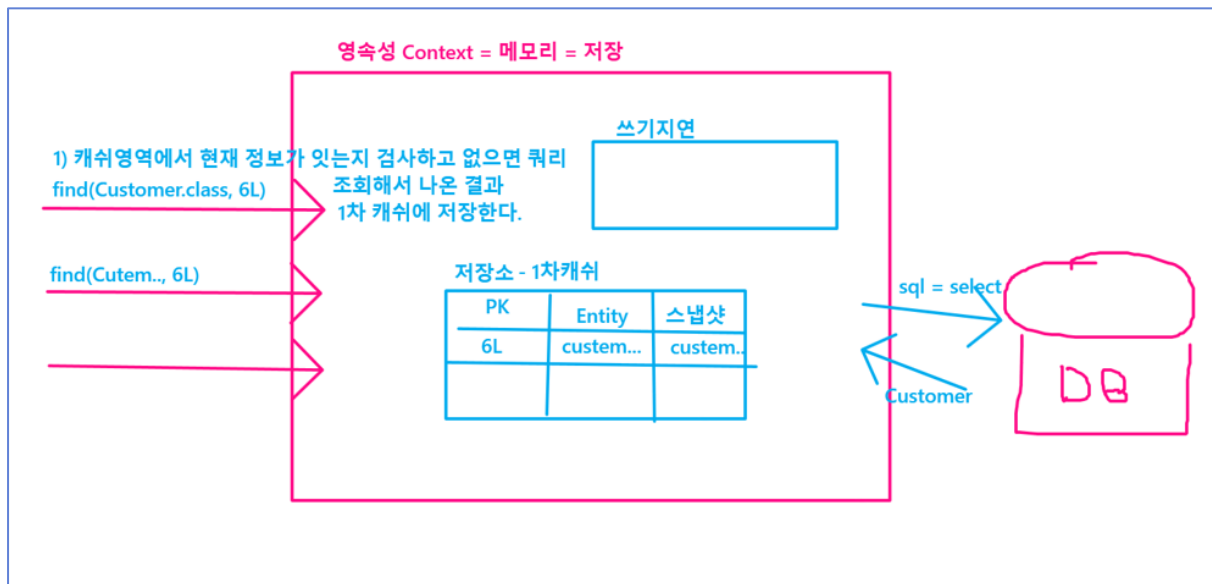
-데이터 등록

1. Java 어플리케이션에서 어떤 엔티티 객체를 생성하여 JPA에게 데이터베이스 저장을 요구하면,
2. 만들어진 엔티티는 1차적으로 영속성 컨텍스트에 저장된다.(1차 캐시) 그리고, 저장한 엔티티를 데이터베이스에 저장하기 위한 쿼리문을 생성시켜 쓰기 지연 SQL 저장소에 저장한다. 계속해서 엔티티를 넘기면 엔티티들과 쿼리문들은 차곡차곡 영속성 컨텍스트에 저장된다.
3. Java 어플리케이션에서 커밋 명령이 내려지면 영속 컨텍스트에는 자동으로 flush()가 호출되고,
4. 영속성 컨텍스트의 변경내용을 데이터베이스와 동기(flush)화 한다. -> SQL 저장소의 쿼리를 실행시킨다.
5. 마지막으로 데이터베이스에게 commit 쿼리문을 명령한다.



-데이터조회

1. Java 어플리케이션에서 JPA에게 데이터베이스 조회를 부탁하면, 1차적으로 영속성 컨텍스트에서 엔티티를 찾는다.
2. 있으면 Java 어플리케이션에 엔티티를 넘긴다.
3. 영속성 컨텍스트에 없는 엔티티 조회를 요구하면
4. 쿼리문을 사용해 데이터베이스에서 찾아와
5. 영속성 컨텍스트에 엔티티로 저장하고
6. Java 어플리케이션에 그 엔티티를 넘긴다.



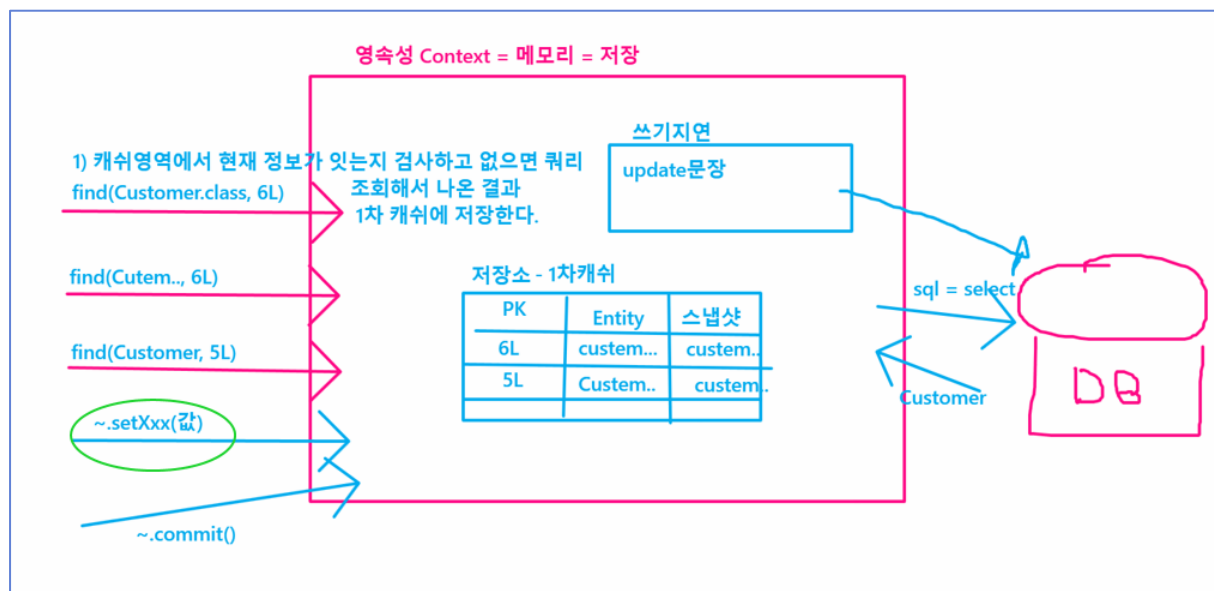
-데이터수정

JPA는 엔티티를 영속성 컨텍스트에 보관할 때, 현재의 상태를 복사해서 저장해 두는데, 이것을 스냅샷이라 한다.

1. Java 어플리케이션에서 커밋 명령이 들어오면, 영속 컨텍스트에는 자동으로 flush()가 호출되고,
2. 엔티티와 스냅샷을 비교해서 변경된 엔티티를 찾는다.

3. 변경된 엔티티가 있으면 데이터베이스에 변경사항을 저장하기 위해 쿼리를 생성하고,
4. 영속성 컨텍스트의 변경내용을 데이터베이스와 동기(flush)화 한다(SQL 저장소의 쿼리를 실행 시 킨다).
5. 마지막으로 데이터베이스에게 commit 쿼리문을 명령한다.

이렇게 엔티티의 변경사항을 데이터베이스에 자동으로 반영하는 기능을 변경감지(Dirty Checking)라 한다.



```

public class MainApp {
    public static void main(String[] args) {
        System.out.println("***JPA 시작하기 *****");
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("JPAProject");

        EntityManager em = emf.createEntityManager();
        EntityTransaction et = em.getTransaction();
        et.begin();

        Customer customer = em.find(Customer.class, 3L);
        customer.setAge(12);
        customer.setUserName("가현");

        et.commit();

        em.close();
        emf.close();
        System.out.println("***JPA 끝 *****");
    }
}

```

- 데이터 삭제

Java 어플리케이션에서 엔티티 삭제 명령이 들어오면, 엔티티를 찾고 쓰기 지연 SQL 저장소에 delete 쿼리를 생성한다.

그리고 Java 어플리케이션에서 커밋 명령이 들어오면, 자동으로 flush()가 호출되고,

영속성 컨텍스트의 변경내용을 데이터베이스와 동기(flush)화 한다(SQL 저장소의 쿼리를 실행시킨다).

마지막으로 데이터베이스에게 commit 쿼리문을 명령한다

```

14 public class MainApp {
15     public static void main(String[] args) {
16         System.out.println("***JPA 시작하기 *****");
17         EntityManagerFactory emf = Persistence.createEntityManagerFactory("JPAProject");
18
19         EntityManager em = emf.createEntityManager();
20         EntityTransaction et = em.getTransaction();
21         et.begin();
22
23         Customer customer = em.find(Customer.class, 3L);
24
25         //삭제
26         em.remove(customer);
27
28         et.commit();
29
30         em.close();
31         emf.close();
32         System.out.println("***JPA 끝 *****");
33     }
34 }
35
36 }

```

트랜잭션 관리

-JPA를 사용하면, 항상 트랜잭션 안에서 데이터를 변경(CUD)해야 한다.

트랜잭션 없이 데이터를 변경하면 예외가 발생한다.

트랜잭션을 시작하려면, 엔티티 매니저에서 트랜잭션 API를 받아와야 한다.

```

EntityTransaction tx = em.getTransaction(); //트랜잭션 기능 획득

try {
    tx.begin(); //트랜잭션 시작
    logic(em); //비즈니스 로직
    tx.commit(); //트랜잭션 커밋
} catch (Exception e) {
    e.printStackTrace();
    tx.rollback(); //트랜잭션 롤백
}

EntityTransaction tx = em.getTransaction();
엔티티 매니저로부터 트랜잭션을 받아온다.
tx.commit();
비즈니스 로직이 정상 동작하면, 트랜잭션을 커밋한다.
tx.rollback();
예외가 발생하면 트랜잭션을 롤백한다.

```

영속성 컨텍스트의 특징

- 캐시 사용 엔티티 조회시 영속성 컨텍스트에 존재하면 바로 리턴, 없으면 데이터베이스 조회 후 리턴.
- 동일성(==)보장 조회 시 항상 동일한 엔티티 객체를 리턴(참조값이 같음)
 - 트랜잭션을 지원하는 쓰기 지연(Transactional write-behind) 트랜잭션 커밋 될때까지 내부 쿼리 저장소에 모아뒀다가 한 번에 실행
- 변경감지(Dirty Checking) 엔티티의 스냅샷을 유지하면서 엔티티의 변경사항을 체크한다.
- 지연로딩(Lazy Loading) 지원 연관된 엔티티를 모두 불러오는 것이 아니라, 실제 호출될 때 로딩되도록 지원(프록시 객체 사용)

JPA 객체지향 쿼리 언어

: JPA는 다양한 쿼리 방법을 지원한다.

- JPQL
- JPA Criteria
- QueryDSL
- NativeQuery

1) JPQL(Java Persistence Query Language)

SQL을 추상화한 객체 지향 쿼리 언어이다. SQL과 문법이 유사하다.

SELECT, FROM, WHERE, GROUP BY, HAVING, JOIN 지원

JPQL은 엔티티 객체를 대상으로 쿼리를 수행한다.

SQL은 데이터베이스 테이블을 대상으로 쿼리를 수행

SQL을 추상화 했기 때문에 특정 DB 문법에 의존하지 않는다.

```
String sql="select m from Member m where m.username like '%jang%'"
```

```
List result = em.createQuery( sql , Member.class).getResultList();
```

2) JPA Criteria

JPA 공식 기능이며 JPQL 빌더 역할을 한다.

Java코드로 쿼리문을 작성함.

Java로 작성했기에 동적쿼리로 작성하기 쉽고, 코드가 잘못되면 컴파일 오류가 나기에 쉽게 고치는 장점이 있다.

너무 복잡하고 실용성이 없어서 굳이 사용한다면 이것 대신에 QueryDSL 사용을 권장한다.

3) QueryDSL

문자열이 아닌 Java코드로 작성 할수 있도록 도와주는 빌더 API이다.

컴파일 시점에 문법 오류를 찾을 수 있다.

Criteria에 비해 편리하고 실용적이다.

SQL과 비슷하여 단순하고 쉬우며 실무 사용에 권장된다.

동적쿼리 지원



```
JPAQuery query = new JPAQuery(em);
QMember m = QMember.member;

List<Member> list = query.from(m)
    .orderBy(m.age.desc())
    .offset(10)
    .limit(20)
    .list(m);
```

JPA 기본키 매핑

기본키(primary key)를 매핑하는 방법은 2가지로 직접 할당과 자동 생성이 있다.

직접 할당은 엔티티에 @Id 어노테이션만 사용해서 직접 할당하는 것이다.

자동 생성은 엔티티에 @Id와 @GeneratedValue를 추가하고 원하는 키 생성 전략을 선택한다.

자동 생성 같은 경우에는 MySQL의 AUTO_INCREMENT 같은 기능으로 생성된 값을 기본키로 사용 하는 것이다.

IDENTITY : 기본키 생성을 DB에 위임한다.

SEQUENCE : DB 시퀀스를 사용해서 기본키를 할당한다.

TABLE : 키 생성 테이블을 사용한다.

AUTO : 선택한 DB에 따라 IDENTITY, SEQUENCE, TABLE 중 하나를 자동으로 선택한다.

- 기본키 자동 생성 전략 - IDENTITY

기본키 생성을 DB에 위임하는 전략이다.

MySQL, PostgreSQL, SQL Server, DB2에서 사용한다.

(MySQL의 AUTO_INCREMENT)

JPA는 보통 트랜잭션 커밋 시점에 INSERT SQL 실행하는데 이 전략은 DB에 먼저 INSERT SQL을 실행해야 기본키를 알 수 있어서 em.persist() 시점에 즉시 INSERT SQL 실행하고 DB에서 식별 자를 조회하게 된다.

```
@Entity
public class Member {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```


기본키 자동 생성 전략 - SEQUENCE

유일한 값을 순서대로 생성하는 특별한 데이터베이스 오브젝트이다.

이 시퀀스를 사용해서 기본키를 생성하게 된다.

시퀀스를 지원하는 Oracle, PostgreSQL, DB2, H2 Database에서 사용할 수 있다.

```
@Entity
public class Customer {
    @Id //pk
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "customer_id_seq")
    @SequenceGenerator(
        name = "customer_id_seq", //식별자생성기이름
        sequenceName = "customer_id_seq", //데이터베이스에 등록되어 있는 시퀀스이름
        initialValue = 1, //처음 시작하는 수 지정
        allocationSize = 1 //시퀀스 한 번 호출에 증가하는 수
    )

    private Long id;
```

IDENTITY 전략과 비슷하지만 내부 동작 방식은 다르다.

시퀀스 전략은 em.persist()를 호출할 때 먼저 DB 시퀀스를 사용해서 식별자를 조회한다.

그리고 조회한 식별자를 엔티티에 할당한 후에 엔티티를 영속성 컨텍스트에 저장한다.

이후 트랜잭션을 커밋해서 플러시가 일어나면 엔티티를 DB에 저장하게 된다.

JPA 연관관계 매핑 기초

: 객체의 참조와 테이블의 외래 키 관계를 매핑하는 것

1) 객체와 테이블의 연관관계 방식

-> 참조(주소)를 사용해서 관계를 맺는다.

2) 테이블의 연관관계 방식

-> 외래키를 사용해서 관계를 맺는다.

방향(단방향, 양방향)**【 테이블 】**

외래키 하나로 조인을 사용해서 양방향으로 쿼리가 가능하므로,
사실상 방향이라는 개념이 없다.

【 객체 】

참조용 필드를 가지고 있는 연관된 객체를 조회할 수 있다.

단방향 : 한 쪽만 참조하는 것

단방향 예시 - 회원과 팀이 존재하고, 서로 관계가 있을 때 "회원 → 팀" or "팀 → 회원"

양방향 : 양 쪽이 서로 참조하는 것

양방향 예시 "회원 → 팀"과 "팀 → 회원" 둘 다 참조하는 것

차수성**【 다 대 일 】**

Ex) 여러 회원이 한 팀에 속한다. 그러므로, 회원과 팀은 "다대일" 관계이다.

【 일 대 다 】

Ex) 한 팀에 여러 회원이 속한다. 그러므로, 팀과 회원은 "일대다" 관계이다.

연관관계의 주인

객체를 양방향 연관관계로 만들면 연관관계의 주인을 정해야 한다.

연관관계의 주인이란?

JPA는 두 객체 연관관계에서 하나의 객체를 정해서 DB 외래키를 관리한다. 이 객체를 연관관계의 주인이라고 한다.

'외래키를 가진 테이블과 매핑되는 엔티티'가 외래키를 관리하는 것이 효율적이다.

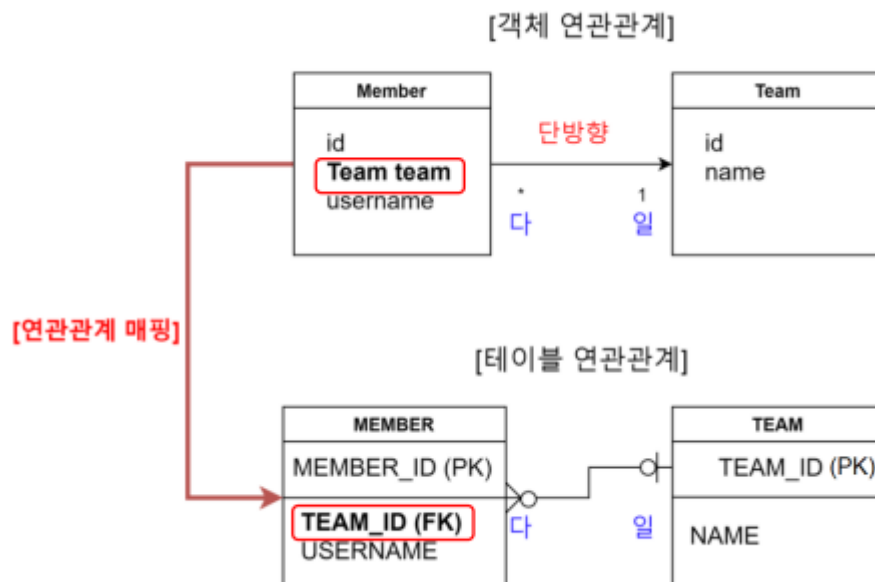
그러므로, '외래키를 가진 테이블과 매핑되는 엔티티'가 연관관계의 주인이 된다.

즉, 해당 엔티티의 외래키 필드가 연관관계의 주인이다.

☞ 다 대 일의 단방향 연관관계

DB 테이블의 일(1), 다(N) 관계에서 외래키는 항상 다 쪽에 있다. 그러므로 객체 양방향 관계에서 연관관계의 주인은 항상 다쪽이다.

객체간의 연관관계에서는 단방향 관계이므로 member → team 조회는 가능하지만 반대의 경우는 할 수 없다. Member.team 필드를 통해서 팀을 알 수 있지만, 반대로 팀은 회원을 알 수 없다.



JPA를 사용해서 객체 연관관계와 테이블 연관관계를 매핑하면 위 그림처럼 된다. 회원 객체의 Member.team 필드와 회원 테이블의 MEMBER.TEAM_ID 외래키 컬럼이 매핑되는 것이다

```

- 회원 엔티티
@Getter
@Setter
@ToString
@NoArgsConstructor
@Entity
@Table(name="membertbl")
public class Member {
    @Id
    @Column(name = "MEMBER_ID")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String username;
    // 연관관계 매핑
    @ManyToOne
    @JoinColumn(name = "TEAM_ID")
    private Team team;

    @OneToOne
    @JoinColumn(name = "LOCKER_ID")
    private Locker locker;

    public Member(String username, Team team, Locker locker) {
        super();
        this.username = username;
        this.team = team;
        this.locker = locker;
    }
}

- 팀 엔티티
@Getter
@Setter
@ToString
@NoArgsConstructor
@AllArgsConstructor
@Entity
public class Team {
    @Id
    @Column(name = "TEAM_ID")
    private String id;
    private String name;
}

```

@ManyToOne 이란?

다대일(N:1) 관계라는 매핑 정보이다. 테이블 연관관계를 보면 회원과 팀의 관계가 N:1로 표현되어 있다

속성	기능	기본값
optional	false로 설정하면 연관된 엔티티가 항상 있어야 한다.	TRUE
fetch	글로벌 페치 전략을 설정한다.	@ManyToOne=FetchType.EAGER @OneToMany=FetchType.LAZY
cascade	영속성 전이 기능을 사용한다.	
targetEntity	연관된 엔티티의 타입 정보를 설정한다. 이 기능은 거의 사용안함.	

@JoinColumn 이란?

@JoinColumn은 외래키를 매핑할 때 사용한다. name 속성에는 매핑할 외래키 이름을 지정한다. 회원과 팀 테이블은 TEAM_ID 외래키로 연관관계를 맺으므로 이 값을 지정한다.

생략할 수 있다. **생략한다면** 외래키를 찾을 때 기본 전략을 사용하게 된다. 기본 전략 : 필드명 + _ + 참조하는 테이블의 기본키 컬럼명 (e.g. team_TEAM_ID 외래키를 사용

속성	기능	기본값
name	매핑할 외래키 이름	필드명 + _ + 참조하는 테이블의 기본키 컬럼명
referencedColumnName	외래키가 참조하는 대상 테이블명의 컬럼명	참조하는 테이블의 기본키 컬럼명
foreignKey(DDL)	외래키 제약조건 직접 지정. 테이블 생성할 때만 사용.	
unique, nullable, insertable, updatable, columnDefinition, table	@Column 속성과 같다.	

☞ 일 대 일의 연관관계

일대일 관계는 양쪽이 서로 하나의 관계만 가진다.

ex) 회원은 하나의 사물함만 사용하고, 사물함도 하나의 회원에 의해서만 사용된다.

이때, '주 테이블 = 회원', '대상 테이블 = 사물함'이다

테이블은 주 테이블이든 대상 테이블이든 외래키 하나만 있으면 양쪽으로 조회할 수 있다.

일대일 관계는 주 테이블이나 대상 테이블 둘 중 어느 곳이나 외래키를 가질 수 있으며 주 테이블

불이 나 대상 테이블 중, 누가 외래키를 가질지 선택해야 한다.

주 테이블에 외래키가 있는 경우 주 객체가 대상 객체를 참조하는 것처럼, 주 테이블에 외래키를 두고 대상 테이블을 참조한다. 외래키를 객체 참조와 비슷하게 사용할 수 있다. 주 테이블이 외래키를 가지고 있으므로, 주 테이블만 확인해도 대상 테이블과 연관관계가 있는지 알 수 있다.

주 테이블에 외래키가 있으면 JPA에서 좀 더 편리하게 매핑할 수 있다.

